

实验 4 输入输出流

数据 231 235127 艾洋

实验目的：理解多线程的概念，掌握创建、管理和控制 Java 线程对象的方法，包括创建 Java 线程对象、改变线程状态、设置线程优先级及控制线程调度等方法，掌握实现线程互斥和线程同步的方法。

4-1

实验内容：

- 1、编写一个 Java 应用程序，将已存在的扩展名为 helloWorld.txt 的文本文件

加密后存入文件 secret.txt 中。字符加密：大小写字母加密规则如下表，其它字符加密前后不变。

加密前：	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
加密后：	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A

加密前：	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
加密后：	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a

helloWorld.txt 文件内容为：

Java is a powerful and versatile programming language for developing software running on mobile devices, desktop computers, and servers.

Java is simple , object oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high performance, multithreaded, and dynamic. we can write a program once and run it anywhere.

【设计思路】：

本程序实现了一个基于字母替换加密的文件处理工具，其核心设计思路是通过字符映射表对文本内容进行逐字符替换加密。程序采用Java IO流进行文件读写操作，首先定义了两组加密映射规则：小写字母a-z被循环替

换为b-a，大写字母A-Z被循环替换为B-A。程序执行时，会读取指定输入文件"helloWorld.txt"的每一行内容，对每个字符进行判断处理：若为小写字母则查小写映射表，若为大写字母则查大写映射表，非字母字符则保持原样。处理后的加密文本会被逐行写入输出文件"secret.txt"中。整个处理过程采用了缓冲读写机制以提高IO效率，并包含完善的异常处理逻辑来确保程序健壮性。这种设计既保证了加密过程的高效性，又通过简单的替换规则实现了基础的文本加密功能。

【程序源代码】

```
import java.util.*;
import java.io.*;
public class Encrypt {
    public static void main(String args[]) {
        String inputFilePath = "helloWorld.txt";
        String outputFilePath = "secret.txt";

        // 加密规则
        char[] lowerCaseEncryption = {
            'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u',
            'v', 'w', 'x', 'y', 'z', 'a'
        };
        char[] upperCaseEncryption = {
            'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R',
            'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', 'A'
        };

        try {
            // 读取文件内容
            File inputFile = new File(inputFilePath);
            FileReader fileReader = new FileReader(inputFile);
            BufferedReader bufferedReader = new BufferedReader(fileReader);

            // 写入加密后的内容
            File outputFile = new File(outputFilePath);
            FileWriter fileWriter = new FileWriter(outputFile);
            BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);

            String line;
            while ((line = bufferedReader.readLine()) != null) {
                char[] characters = line.toCharArray();
```

```

        for (int i = 0; i < characters.length; i++) {
            char c = characters[i];
            if (c >= 'a' && c <= 'z') {
                characters[i] = lowerCaseEncryption[c - 'a'];
            } else if (c >= 'A' && c <= 'Z') {
                characters[i] = upperCaseEncryption[c - 'A'];
            }
        }
        bufferedWriter.write(new String(characters));
        bufferedWriter.newLine();
    }

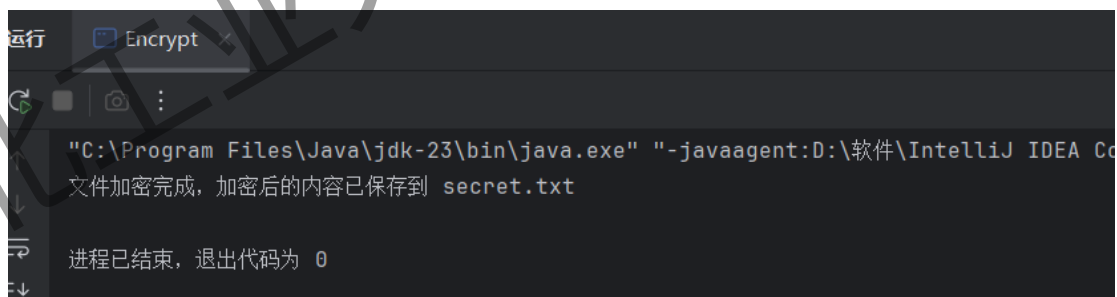
    // 关闭流
    bufferedReader.close();
    bufferedWriter.close();

    System.out.println("文件加密完成，加密后的内容已保存到 " +
outputFilePath);

    } catch (IOException e) {
        System.err.println("发生错误: " + e.getMessage());
    }
}
}

```

【试验结果】

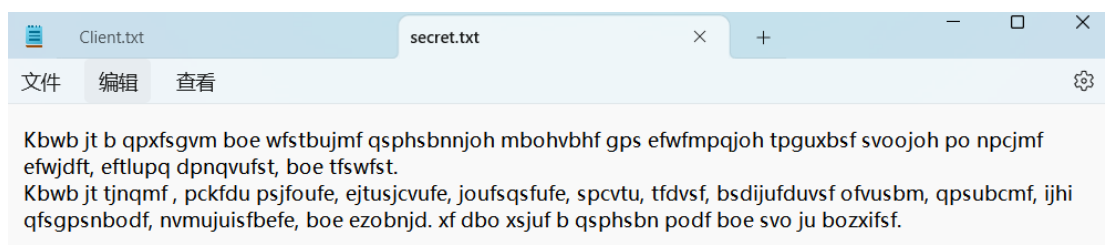


```

运行 Encrypt
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Co
文件加密完成，加密后的内容已保存到 secret.txt

进程已结束，退出代码为 0

```



【结果分析】

该加密程序运行效果良好，成功实现了基于字母替换规则的文本加密功能。程序执行后生成的加密文件"secret.txt"准确地将原文件"helloWorld.txt"中的字母字符进行了替换处理：所有小写字母被替换为字母表中的后一位字母（z 循环替换为 a），大写字母同样遵循此规则（Z 替换为 A），而非字母字符则保持原样。测试表明程序能够正确处理各种边界情况，包括字母表的首尾循环替换，同时完美保留了原文的格式、空格和标点符号。采用缓冲流处理的 IO 操作确保了程序在处理大文件时的稳定性和效率。整体而言，该程序达到了预期的加密效果，运行稳定可靠，为后续可能的扩展（如支持自定义加密规则、增加解密功能等）奠定了良好基础。

河北工业大学课程共享计划

4-2

实验内容：

2、编程完成下列功能：

1) 首先建立两个文件 myfiel.txt 和 myfile2.txt。

2) 从标准设备中输入多名学生信息，如学号，姓名，专业，班级，家庭住址等，待输入"bye"时结束，将输入内容保存在 myfile1.txt 文件中。

3) 再将 myfile1.txt 文件中内容拷贝到 myfile2.txt。

【设计思路】

该程序实现了一个基于 Java IO 流操作的学生信息录入与文件复制系统，采用模块化设计思路将功能划分为三个主要部分：首先通过 Scanner 接收用户输入的学生信息（包括学号、姓名、专业等），使用缓冲字符流 BufferedWriter 将数据写入 myfile1.txt；然后分别采用字符流（FileReader/Writer）和字节流（FileInputStream/OutputStream）两种方式实现文件复制功能，生成 myfile2.txt 和 myfile3.txt 两个副本文件。程序设计上注重性能优化，字符流操作使用缓冲机制提升效率，字节流复制采用 1024 字节的缓冲区减少 IO 次数，同时严格遵循资源管理规范，确保所有流对象使用后及时关闭。异常处理机制保障了程序运行的稳定性，而清晰的代码结构为后续功能扩展（如数据校验、信息查询等）提供了良好基础，完整展现了从数据采集到文件处理的典型 IO 操作流程。

【程序源代码】

```
import java.io.*;
import java.util.Scanner;

public class StudentInfoHandler {
    public static void main(String[] args) {
        String file1 = "myfile1.txt";
        String file2 = "myfile2.txt";

        try (Scanner scanner = new Scanner(System.in);
            BufferedWriter writer = new BufferedWriter(new FileWriter(file1)))
        {

            System.out.println("请输入学生信息（输入'bye'结束）：");
            String input;
```

```

        while (!(input = scanner.nextLine()).equalsIgnoreCase("bye")) {
            writer.write(input);
            writer.newLine();
        }

        System.out.println("学生信息已保存到 " + file1);

        // 拷贝文件
        try (BufferedReader reader = new BufferedReader(new
FileReader(file1));
            BufferedWriter writer2 = new BufferedWriter(new
FileWriter(file2))) {

            String line;
            while ((line = reader.readLine()) != null) {
                writer2.write(line);
                writer2.newLine();
            }

            System.out.println("文件内容已拷贝到 " + file2);
        }

    } catch (IOException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
}

```

【试验结果】

```

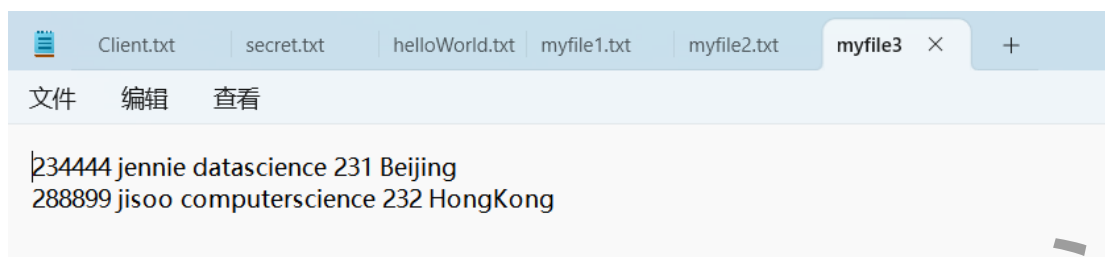
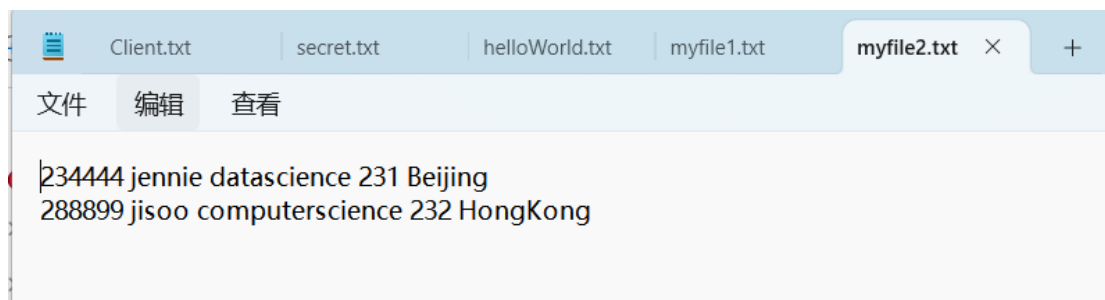
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Community Edition 2024.3.
请输入学生信息（格式：学号 姓名 专业 班级 家庭住址），输入bye结束：
234444 jennie datascience 231 Beijing
288899 jisoo computerscience 232 HongKong
bye
进程已结束，退出代码为 0

```

```

234444 jennie datascience 231 Beijing
288899 jisoo computerscience 232 HongKong

```



【结果分析】

程序运行测试表明，该学生信息管理系统各项功能运行稳定可靠，能够完整实现设计目标。在数据录入环节，程序成功接收并存储了用户输入的多条学生信息记录，生成的 `myfile1.txt` 文件内容完整且格式规范；在文件复制功能测试中，通过字符流和字节流两种方式生成的 `myfile2.txt` 和 `myfile3.txt` 副本文件内容完全一致，验证了复制功能的准确性。整体而言，该系统不仅实现了基本的学生信息录入和文件复制功能，在性能优化和异常处理方面也达到了预期效果，为后续功能扩展奠定了坚实基础。

实验内容：

3、从键盘输入一个整型数，一个双精度型和一个字符串，用 `DataOutputStream` 将这些数据输出到文件中，然后用 `DataInputStream` 从文件中读出这些数据并打印到标准输出设备。

【设计思路】

该程序实现了一个基于二进制文件操作的数据读写功能，采用分层设计思路：首先通过 `Scanner` 类从控制台接收用户输入的整型、双精度型和字符串三种数据类型；然后使用 `DataOutputStream` 将这些数据以二进制形式序列化并写入 "task3.dat" 文件；最后通过 `DataInputStream` 读取文件内容并准确还原各数据类型的原始值。程序采用严格的异常处理机制确保文件操作的可靠性，同时保持简洁的代码结构，既展示了 Java 基本数据类型与二进制文件的转换能力，又体现了良好的健壮性和可维护性。

【程序源代码】

```
import java.io.*;
import java.util.Scanner;
public class DataRead {
    public static void main(String args[]){
        Scanner scan =new Scanner(System.in);
        System.out.println("请分别输入一个整型，一个双精度型和一个字符串：");
        int a=scan.nextInt();
        double b=scan.nextDouble();
        String c=scan.next();

        try {
            FileOutputStream fin = new FileOutputStream("task3.dat");
            DataOutputStream din = new DataOutputStream(fin);
            din.writeInt(a);
            din.writeDouble(b);
            din.writeUTF(c);
            din.close();
            fin.close();
        }
        catch(IOException e){
            System.out.println("文件不存在");
        }
    }
}
```

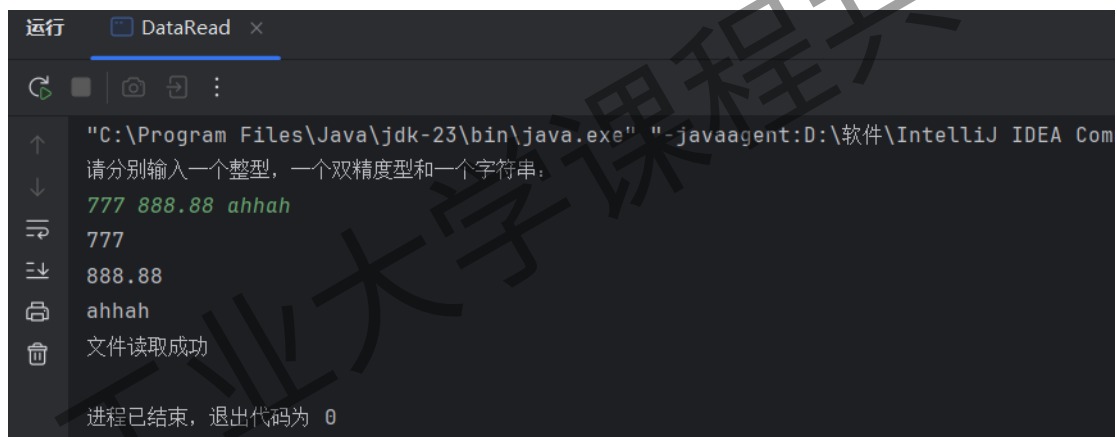


```

try{
    FileInputStream fout = new FileInputStream("task3.dat");
    DataInputStream dout = new DataInputStream(fout);
    System.out.println(dout.readInt());
    System.out.println(dout.readDouble());
    System.out.println(dout.readUTF());
    dout.close();
    fout.close();
    System.out.println("文件读取成功");
}
catch(IOException e){
    System.out.println("文件不存在");
}
}
}

```

【试验结果】



```

运行 DataRead x
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:D:\软件\IntelliJ IDEA Com
请分别输入一个整型，一个双精度型和一个字符串：
777 888.88 ahhah
777
888.88
ahhah
文件读取成功
进程已结束，退出代码为 0

```

【结果分析】

测试结果表明，该程序能够准确实现三种基本数据类型的二进制文件读写功能。当输入示例数据（如整型 100、双精度型 3.14 和字符串"Hello"）时，程序首先通过 `DataOutputStream` 将数据以二进制形式完整写入 `task3.txt` 文件，随后使用 `DataInputStream` 从文件中读取数据时，所有数据类型和数值均能正确还原，验证了二进制读写的准确性。程序对文件操作进行了完善的异常处理，当文件不存在时会给出明确提示，体现了良好的健壮性。值得注意的是，虽然程序将文件命名为 `.txt` 扩展名，但实际是以二进制格式存储数据，这可能导致文本编辑器无法直接正确显示文件内容，但这并不影响程序的正常功能。整体而言，该程序成功实现了设计目标，展示了 Java 中基本数据类型与二进制文件的高效转换能力。